TapMap — Complete Step-by-Step Build Plan

Autonomous Percussive Defect-Mapping Rover · Group 11 · CSE-AI Batch A · 5-week compressed schedule · v1.0

How to use this document

Every step has a **GATE** — a concrete, testable condition. Do not start the next step until the current gate passes. This is the single most important rule in the plan: a robot built on an unverified layer fails in a way that is very hard to debug later. Steps are grouped into three parallel tracks so three people can work at once:

Track	Owner	Scope
A — Software	Abdul	Raspberry Pi, ROS 2 nodes, localisation, planners, DL training, RViz, PDF report
B — Firmware	Sri Nandan	ESP32 micro-ROS, motor PID, encoder counting, I²S audio capture, solenoid gating
C — Hardware	Bharadwaj	Chassis, wiring, power tree, tapper mount, demo panel, dataset recording

Schedule overview

Week	Theme	Exit milestone
0	Review prep (now → Sept 9)	Mid review delivered; Pi optionally flashed
1	Bring-up: Pi, ROS 2, micro-ROS, bench tests	Encoder ticks visible as a ROS 2 topic on the Pi
2	Motion + localisation + full wiring	2 m out-and-back error < 2 cm over 10 runs
3	Tapper, gated audio, dataset, DL-1	Live blind test: robot identifies the hollow tile 9 of 10 times
4	Autonomy: discovery, coverage, heat map, DL-2/DL-3	First fully autonomous end-to-end scan with PDF output
5	Hardening, demo, paper	5 consecutive clean scans; demo rehearsed; paper drafted

Blocking dependencies — chase these now

1. The 12 V battery. The lab is holding it until setup is done. It is a Week 2 gate — without it the rover cannot drive untethered. Get a date in writing this week.
2. Solenoid delivery. Expected tomorrow. Needed by Week 3 Step 3.1. No slack issue if it arrives as promised.
3. Demo panel decision. Recommended: lay the tiles at floor level in a corner of the lab, so real walls serve as the ToF boundary reference. A tabletop panel needs a 5 cm raised lip added around the edge.

WEEK 0 — Review preparation (now → 9 September)

Rule for this week: change nothing that could break. The simulation currently works — protect that.

Step 0.1 — Rehearse the live simulation demo (Track A)

On the laptop, from the existing tapmap_ws:

```
colcon build
source install/setup.bash
ros2 launch tapmap sim.launch.py

# second terminal
rviz2 -d src/tapmap/rviz/tapmap.rviz

# third terminal
ros2 action send_goal /scan_area tapmap_interfaces/action/ScanArea \
  "{width: 0.8, height: 0.5, spacing: 0.10, return_home: true}" --feedback
```

Also have `rqt_graph` open in a fourth terminal — the live node graph is the single best visual answer to "where is ROS 2 used?"

GATE: you can run the whole demo cold, from a closed laptop, in under 3 minutes.

Step 0.2 — Optional: flash the Pi before the review (Track A)

If time allows, complete Step 1.1 early. Being able to say "the ROS 2 stack already runs on the robot's onboard computer" is materially stronger than "it runs on my laptop." If anything goes wrong, you still have the laptop demo — nothing is lost.

Step 0.3 — Prepare answers (all)

Likely question	Answer
Why not Nav2?	Nav2 assumes obstacle-rich costmap navigation. On a flat scan surface it is unjustifiable overhead; a purpose-built coverage planner matches the problem. Nav2 is future work for cluttered sites.
Ambient noise will ruin it	Capture is a 100 ms window gated to the strike. The tap transient dominates ambient by a large margin. We also train with hall noise mixed in, and re-tap when SNR is below floor.
Why is navigation not learned?	On a known rectangular grid a learned controller adds training burden and failure modes with zero accuracy gain. Learning is applied where rules genuinely fail — acoustic classification.
How do you know the map coordinates are right?	The rover returns to origin and reports its own position error. If it lands within 2–3 cm of the start mark, odometry held for the whole scan.
Is this just IoT on wheels?	It plans coverage, localises itself by sensor fusion, and produces a spatial occupancy grid. Sense → plan → act → localise → map is the full robotics loop.

WEEK 1 — Bring-up

Goal: every subsystem proves it is alive, independently. No integration yet.

Track A — Software

Step 1.1 — Flash the Raspberry Pi 4

Back up anything on the existing microSD first — confirm with lab staff that wiping it is acceptable. Use Raspberry Pi Imager on your laptop:

```
OS: Ubuntu Server 24.04 LTS (64-bit) ← must be 64-bit, must be Server
In Imager's advanced settings, set BEFORE writing:
  · hostname: tapmap
  · username / password
  · Wi-Fi SSID + password + country IN
  · enable SSH
```

Boot with HDMI and keyboard the first time, confirm login, then find the IP with `ip a`. After that go headless over SSH.

GATE: you can SSH into the Pi from your laptop with the monitor unplugged.

Step 1.2 — Install ROS 2 Jazzy on the Pi

```
sudo apt update && sudo apt upgrade -y
# add the ROS 2 apt repository (official Jazzy instructions for Ubuntu 24.04)
sudo apt install ros-jazzy-ros-base python3-colcon-common-extensions
sudo apt install ros-jazzy-robot-localization ros-jazzy-tf2-ros
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc

# on the LAPTOP install the desktop variant (you need RViz there)
sudo apt install ros-jazzy-desktop
```

Install `ros-jazzy-ros-base` on the Pi, not desktop — the Pi is headless and does not need RViz.

GATE: `ros2 doctor` reports no critical errors on the Pi.

Step 1.3 — Build the existing workspace on the Pi

```
scp -r tapmap_ws tapmap@<pi-ip>:~/
ssh tapmap@<pi-ip>
cd ~/tapmap_ws && colcon build && source install/setup.bash
ros2 launch tapmap sim.launch.py
```

Then on the laptop, with both machines on the same Wi-Fi and the same ROS_DOMAIN_ID, open RViz and confirm you can see the Pi's topics. This proves the distributed ROS 2 graph works across machines — which is exactly what the final system does.

GATE: RViz on the laptop shows the simulated heat map being built by nodes running on the Pi.

Track B — Firmware

Step 1.4 — micro-ROS toolchain and first node

Install PlatformIO (VS Code extension is easiest). Create the firmware project, add the micro_ros_platformio library, and write a minimal node that publishes a counter over USB serial. On the Pi, run the agent:

```
# on the Pi — install and run the micro-ROS agent
sudo apt install ros-jazzy-micro-ros-agent # or build from source if unavailable
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0 -b 115200
```

GATE: `ros2 topic list` on the Pi shows a topic published by the ESP32 over the USB cable.

Step 1.5 — Encoder counting with hardware PCNT

Wire both encoder channels to the ESP32's input-only pins (GPIO 34/35 and 36/39) and use the ESP32's hardware pulse counter units — not software interrupts. PCNT counts in hardware with zero CPU load, which matters because the audio DMA cannot be starved later. Publish a custom WheelTicks message at 50 Hz.

GATE: `ros2 topic echo /wheel_ticks` — turning a wheel by hand one full revolution changes the count by ~700, and the count does not drift when the wheel is still.

Track C — Hardware

Step 1.6 — Bench-test every module individually

Before anything is mounted, power up each module on the breadboard and confirm it responds: ESP32 blink, MPU6050 I²C address 0x68 detected, VL53L0X 0x29, OLED 0x3C, motors spin on the L298N from a bench supply. Log which ones work. A dead module found now costs an hour; found in Week 4 it costs a weekend.

Step 1.7 — Set the XL4015 output BEFORE connecting the Pi

This step can destroy the Raspberry Pi if skipped. The XL4015 ships at an arbitrary output voltage.

1. Connect the 12 V source to the XL4015 input, with **nothing** on the output.
2. Put a multimeter across the output terminals.
3. Turn the blue trimmer potentiometer until it reads 5.0–5.1 V.
4. Only now connect the Pi.

GATE: output measured at 5.0–5.1 V with the meter, twice, before the Pi is ever attached.

Step 1.8 — Mechanical assembly

Strip the BO motors from the lab chassis. Mount the two N20 encoder motors on the centre axis with zip ties or hot glue (brackets were out of stock — zip ties through existing chassis holes are rigid enough if pulled tight). Fit the 43 mm wheels, the ball caster at the rear, and optionally leave two old wheels on as free-spinning dummies for the rover look. Battery and Pi go low and centred over the wheel axis; ESP32 and drivers on the upper deck.

GATE: chassis rolls straight when pushed by hand; nothing shifts when you shake it.

WEEK 2 — Motion and localisation

This is the most important week in the project. Everything downstream depends on odometry being trustworthy.

Step 2.1 — Motor PID and /cmd_vel (Track B)

Add a 100 Hz control task on ESP32 core 1: read encoder deltas, compute wheel velocities, run a PI(D) loop per wheel, write PWM to the L298N. Subscribe to /cmd_vel and convert (linear.x, angular.z) into wheel setpoints using the differential-drive model. Add a 200 ms deadman: if no /cmd_vel arrives, stop the motors.

GATE: `ros2 topic pub /cmd_vel` **drives the robot; unplugging the Pi stops it within 250 ms.**

Step 2.2 — Odometry and TF (Track A)

Write base_controller: subscribe /wheel_ticks, apply differential-drive kinematics, publish nav_msgs/Odometry on /odom and broadcast the odom → base_link transform. Calibrate wheel_radius and wheel_separation empirically — command a 1.0 m straight drive, measure the real distance, adjust the radius until they match; command a 360° spin, adjust separation until it lands square.

GATE: commanded 1.0 m produces 1.0 m ± 2 cm on the floor, measured with a tape.

Step 2.3 — IMU fusion (Track A + B)

Publish /imu/data_raw at 100 Hz from the ESP32. On the Pi, filter it to /imu/data and configure robot_localization's EKF to fuse wheel odometry (x, y velocity) with IMU yaw. Encoders are excellent at distance and poor at heading; the gyro is the reverse. Fusing them is why rows stay straight.

GATE: ten consecutive 180° turns land within 3° of target, measured against a floor mark.

Step 2.4 — Complete the TF tree (Track A)

Publish static transforms for imu_link, tof_link, mic_link and — critically — **tapper_link** at the real measured offset of the striker tip from base_link (roughly 6 cm forward). Every tap will be stamped at this frame, not at robot centre. Getting this wrong shifts the entire defect map.

GATE: `ros2 run tf2_tools view_frames` **shows the complete tree with no disconnected frames.**

Step 2.5 — Full wiring and power tree (Track C)

Permanent wiring: 12 V battery → master rocker → E-stop (in the actuator branch only) → L298N and the solenoid rail; 12 V → XL4015 → 5 V → Pi; ESP32 powered from the Pi's USB. Twist the motor leads. Keep the I²S mic wiring short and away from motor leads. Star-ground at the buck input. Every module on removable connectors — nothing soldered captive.

GATE: robot drives untethered on battery for 10 minutes with no resets and no rail dips.

Step 2.6 — Build the demo panel (Track C)

Lay 6–8 tiles on plywood or directly on a flat floor area. Bond 5–7 of them fully with adhesive or full-coverage double-sided foam tape. For the hollow bay, use only four small corner dots of tape — this creates a genuine air gap and a genuinely different acoustic signature. **Record which bay is hollow** and keep that note; it is your ground truth for every accuracy claim you will make.

GATE: tapping the hollow bay with a coin sounds clearly different from its neighbours — to your ear, before any robot is involved.

WEEK 2 EXIT MILESTONE — do not proceed until this passes

Drive 2 m out and 2 m back, ten times. Measure how far from the start mark the robot finishes each time. Ninety-five percent of runs must land within 2 cm. If they do not, fix it here: check for wheel slip (add silicone bands), recalibrate wheel radius and separation, verify the EKF is actually fusing (not just passing odometry through). A map built on 10 cm of drift is worthless, and no amount of good classification will save it.

WEEK 3 — Tapper, audio capture, dataset, DL-1

Step 3.1 — Solenoid drive stage (Track B + C)

Build the MOSFET stage on perfboard: ESP32 GPIO 23 → 100 Ω series resistor → IRLZ44N gate, 10 kΩ gate pulldown, solenoid between 12 V and the drain, 1N5822 flyback diode across the coil (cathode to 12 V), 1000 µF reservoir capacitor across the 12 V rail close to the solenoid. The diode is not optional — without it the coil's collapse spike will destroy the ESP32.

GATE: a 30 ms pulse fires the plunger audibly and repeatably; the 5 V rail dips less than 100 mV during the strike.

Step 3.2 — Tapper mounting (Track C)

Mount the solenoid vertically at the front-centre of the chassis through a cutout, on rubber grommets so the recoil does not ring the chassis and contaminate the IMU. Fit the rubber striker tip (an eraser or chair-leg cap works) — bare steel is loud, damages tiles, and narrows the useful spectrum. Set the stroke so the tip protrudes ~3 mm below the deck at full extension and clears the surface when retracted.

GATE: 50 consecutive taps with no chassis hop, no loosening, no tip damage.

Step 3.3 — Gated I²S audio capture (Track B)

This is the highest-skill step in the firmware. Configure I²S with DMA at 16 kHz, 16-bit mono from the INMP441. Implement the /tapper/fire service so that one call: (1) energises the solenoid for 30 ms, (2) opens a 100 ms DMA capture window timestamped to the trigger, (3) publishes the buffer on /audio/tap_window. The gating is what makes ambient noise a non-issue — you only ever listen during the strike.

GATE: trigger-to-audio jitter under 2 ms; captured waveforms of the same surface look consistent tap after tap.

Step 3.4 — Record the dataset (Track C, with A)

Write a small recorder script that repeatedly calls /single_tap and saves each window as a WAV with a label. Record on the demo panel:

· ≥300 taps on bonded tiles, varying position across several tiles
· ≥300 taps on the hollow bay, varying position within it
· repeat a subset with a phone playing hall/crowd noise nearby, so the model sees realistic conditions

This dataset is a genuine research contribution — no public labelled tap-acoustic dataset of this kind exists at scale.

GATE: ≥600 labelled windows saved, balanced across classes, with metadata (surface, position, noise condition).

Step 3.5 — Train and deploy DL-1 (Track A)

Pipeline: window → MFCC (13 coefficients, ~9 frames) → small CNN (<100 k parameters) → softmax over {bonded, debonded}. Augment with gain variation, time shift and noise mixing. Use stratified k-fold. Export to ONNX and run inference in audio_classifier on the Pi (~2 ms per tap). Keep the spectral-rule classifier as a runtime fallback and as the ablation baseline for the paper.

GATE: ≥90% cross-validated accuracy, AND a live blind test — a teammate moves which bay is hollow, robot finds it 9 times out of 10.

WEEK 4 — Autonomy, mapping, DL-2 and DL-3

Step 4.1 — Boundary discovery (Track A)

Write boundary_explorer hosting the /discover_area action. Sequence: drive forward until /tof/range < 0.15 m → record length L → rotate 90° closed-loop on fused yaw → drive until the next wall → record width W → return to origin. Include a max_length / max_width safety parameter so a missed wall reading cannot send the robot across the building.

GATE: placed in a corner of an unfamiliar rectangular space, the reported L × W is within 5 cm of a tape measurement.

Step 4.2 — Mission FSM (Track A)

A mission planner sequences: DISCOVER → CALIBRATE → SCAN → RETURN → REPORT, calling /discover_area then /scan_area and handling cancellation and faults at every stage. This state machine is the single clearest piece of evidence of ROS 2 depth in the whole project.

Step 4.3 — Coverage on real hardware (Track A)

Port coverage_planner from simulation onto the physical robot: boustrophedon waypoints over the discovered rectangle at 8 cm spacing, drive → settle 400 ms → fire tap → await classification → next. Tune settle time against real vibration decay: too short and the chassis is still ringing when the mic opens; too long and a 1 m² scan takes forever.

GATE: a complete unattended raster of the panel, every cell tapped exactly once, no missed rows.

Step 4.4 — Live defect heat map (Track A)

defect_mapper subscribes /tap/classification, does a TF lookup at tapper_link, and writes into the OccupancyGrid: bonded → 0, debonded → round(100 × confidence). Repeat taps on a cell combine as a running average. Publish /defect_grid at 1 Hz and /defect_markers per tap. Confirm in RViz that the red region lands where the hollow bay physically is.

GATE: the heat map's red region matches the recorded ground-truth bay position within one cell.

Step 4.5 — DL-2 surface classifier (Track A + C)

Record calibration taps on a second surface — a concrete floor slab, a different tile type, or a marble sill. Train a lightweight classifier on MFCC statistics to identify the surface from the 10 calibration taps, and have the system load the matching DL-1 weights. This is what turns a one-panel demo into a deployable instrument, and it is a strong answer to "does this generalise?"

GATE: after a 30 s calibration on a surface it was not tuned for, classification accuracy stays above 85%.

Step 4.6 — DL-3 region refiner (Track A)

Train a compact U-Net-lite over the accumulated defect grid to remove isolated false positives and merge scattered hits into contiguous regions, outputting region boundaries and area estimates. Real debonding is spatially contiguous; a lone red cell in a green field is almost always noise.

GATE: refined map has zero isolated single-cell false positives across five test scans.

Step 4.7 — PDF inspection report (Track A)

report_generator renders: the heat map image, a tap-by-tap table with coordinates and confidences, spectrograms of the flagged cells, total defect area in cm², the scan duration, and the return-home error as a stated accuracy metric. Timestamped and saved on the Pi.

WEEK 4 EXIT MILESTONE

Place the robot in a corner. Press start. It discovers the area, calibrates, rasters the surface tapping every 8 cm, builds a live heat map that correctly marks the hollow bay, returns to its start point, reports its position error, and writes a PDF report — with no human intervention at any point.

WEEK 5 — Hardening, demo and paper

Step 5.1 — Repeatability and failure drills

Test	Pass condition
5 consecutive full scans	All five find the defect; no manual intervention; no crashes
E-stop during motion and during a tap	Actuators dead immediately; logic alive; scan resumable
Wi-Fi killed mid-scan	Scan continues (Pi is onboard); laptop reconnects and catches up
Battery soak	≥ 90 min continuous operation; voltage curve logged
Different tile / second surface	After 30 s calibration, still classifies correctly
Ambient noise at ~70 dB	Classification accuracy holds

Step 5.2 — Lock the configuration

Freeze grid spacing, settle time, speed, PID gains and confidence threshold in params.yaml. Tag the repository. From this point, change nothing without re-running the five-scan test.

Step 5.3 — Record the backup demo video

Film a complete successful scan — robot, screen, and the coin-tap verification — while everything is working. If the live demo fails on the day, this is what saves the presentation. Do not skip it.

Step 5.4 — Demo script (5 minutes)

1. Thirty seconds of problem framing: façade collapse, manual tap testing, no record.
2. A judge places the robot in the corner. Press start.
3. It discovers the area, then begins tapping — audible, visible, deliberate.
4. RViz fills in live; the hollow region turns red.
5. Robot returns to its start mark. State the return error out loud.
6. PDF report opens.
7. Hand a judge a coin: "tap it yourself." They hear the hollow spot exactly where the robot marked it.
8. Close on cost: ■3,000 against several lakh.

Step 5.5 — Paper draft (IEEE format)

Title: A Low-Cost Autonomous Rover for Acoustic Percussion-Based Detection and Spatial Mapping of Tile and Concrete Debonding. Sections: introduction and related work (ASTM D4580, impact-echo, commercial NDT) · system design · depth-indexed acoustic method · dataset description · classification results with the rule-based ablation · field validation on the panel · limitations and future work. Report accuracy, F1, false-positive rate per m², localisation error and runtime per m².

FUTURE WORK (kept deliberately short)

Item	Why it is future work, not now
Nav2 + obstacle handling	Needed only for cluttered real sites; adds no value on a flat scan surface
Void depth estimation	Resonance frequency gives depth (f = C/2d) — requires calibrated void samples at known depths
Non-rectangular workspaces	Perimeter tracing for concave rooms; v1 assumes a convex rectangle
Façade-climbing variant	The largest commercial market, but a different adhesion problem entirely
Multi-robot tiling	Domain-bridged fleet sharing one defect grid across a large deck
Cloud dashboard / BIM overlay	Building-health record with inspection-over-time trend analysis

Risk register — what most likely goes wrong

Risk	Early warning	Response
Odometry drift	Week 2 out-and-back error > 2 cm	Silicone tyre bands, recalibrate radius/separation, verify EKF is fusing, add ToF boundary correction
Dataset too clean	99% training accuracy but poor live performance	Re-record with hall noise, more tile positions, vary tap force slightly
Solenoid shakes the robot	IMU spikes after each tap; chassis hops	Stiffer grommets, longer settle time, add nose mass

Risk	Early warning	Response
Battery arrives late	Week 2 gate slips	Bench-test on the Pi adapter; buy 18650 + BMS from the ■2,800 buffer
Pi thermal throttling	Slow inference late in long scans	Add heatsink; classification is only 2 ms so impact is small
Time runs out	Week 4 not started by 7 Oct	Ship DL-1 only, move DL-2 and DL-3 to future work — the core demo still stands alone